

Sprint 2

Índice

Nivel 1-----	2
Ejercicio 1-----	2
Ejercicio 2-----	4
Ejercicio 3-----	5
Ejercicio 4-----	6
Ejercicio 5-----	8
Ejercicio 6-----	8
Ejercicio 7-----	9
Ejercicio 8-----	10
Ejercicio 9-----	13
Ejercicio 10-----	13
Nivel 2-----	14
Ejercicio 1-----	14
Ejercicio 2-----	14
Ejercicio 3-----	15
Ejercicio 4-----	15
Ejercicio 5-----	16

Nivel 1

Ejercicio 1

A partir de los siguientes documentos adjuntos:

- N1-Ex.1_estructura_dades.sql
- N1-Ex.1_dades_introduir.sql

Importa las dos tablas. Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las distintas tablas y variables.

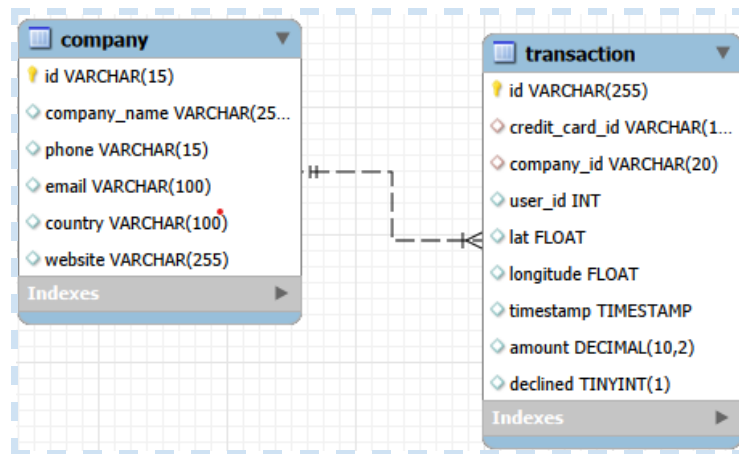
```
3 -- Creamos la base de datos
4 • CREATE DATABASE IF NOT EXISTS transactions;
5 • USE transactions;
6
7 -- Creamos la tabla company
8 • CREATE TABLE IF NOT EXISTS company (
9   id VARCHAR(15) PRIMARY KEY,
10  company_name VARCHAR(255),
11  phone VARCHAR(15),
12  email VARCHAR(100),
13  country VARCHAR(100),
14  website VARCHAR(255)
15 );
16
17 -- Creamos la tabla transaction
18 • CREATE TABLE IF NOT EXISTS transaction (
19   id VARCHAR(255) PRIMARY KEY,
20   credit_card_id VARCHAR(15),
21   company_id VARCHAR(10),
22   user_id INT REFERENCES user(id),
23   lat FLOAT,
24   longitude FLOAT,
25   timestamp TIMESTAMP,
26   amount DECIMAL(18, 2),
27   declined BOOLEAN,
28   FOREIGN KEY (company_id) REFERENCES company(id)
29 );
```

Output

#	Time	Action	Message	Duration / Fetch
17	13:46:24	CREATE DATABASE IF NOT EXISTS transactions	1 row(s) affected	0.000 sec
18	13:46:24	USE transactions	0 row(s) affected	0.000 sec
19	13:46:24	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(255), phone VARCHAR(15), email VARCHAR(100), country VARCHAR(100), website VARCHAR(255))	0 row(s) affected	0.016 sec
20	13:46:24	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, credit_card_id VARCHAR(15) REFERENCES credit_card(id), company_id VARCHAR(10), user_id INT REFERENCES user(id), lat FLOAT, longitude FLOAT, timestamp TIMESTAMP, amount DECIMAL(18, 2), declined BOOLEAN, FOREIGN KEY (company_id) REFERENCES company(id))	0 row(s) affected	0.031 sec

Output

#	Time	Action	Message	Duration / Fetch
100100	20:47:51	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES (...)	1 row(s) affected	0.000 sec
100101	20:47:51	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES (...)	1 row(s) affected	0.016 sec



Esquema de un modelo de tipo estrella con dos tablas; company y transaction, ambas usando ids como primary keys. Dichas tablas tienen una relación 1:N donde se vinculan a través de la foreign key company_id de transaction.

La tabla company incluye el nombre de la compañía, el teléfono, el correo electrónico, el país y la página web como cadena que permite varios tipos de caracteres (VARCHAR).

Transaction incluye la id de la tarjeta de crédito y la compañía como VARCHAR, la id de usuario como número entero (INT), latitud y longitud como decimal aproximado (FLOAT), la fecha y hora de la transacción como TIMESTAMP, la cantidad como decimal exacto (DECIMAL) y si se ha rechazado o no la transacción, como un valor binario (TINYINT).

Ejercicio 2

Utilizando JOIN realizarás las siguientes consultas:

→ Listado de los países que están generando ventas.

```
37 • SELECT DISTINCT country AS paises_generando_ventas
38 FROM company
39 JOIN transaction
40 ON company.id = company_id
41 WHERE declined = 0
42 ;
```

Result Grid

paises_generando_ventas
Germany
Australia
United States
New Zealand
Norway
United Kingdom
Italy
Belgium
Sweden
Ireland

Output

#	Time	Action	Message	Duration / Fetch
100101	20:47:51	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) ...	1 row(s) affected	0.016 sec
100102	20:52:49	SELECT DISTINCT country AS paises_generando_ventas FROM company JOIN transaction ON company.id = ...	15 row(s) returned	0.000 sec / 0.000 sec

→ Desde cuántos países se generan las ventas.

```
46 • SELECT COUNT(DISTINCT country) AS recuento_paises_generando_ventas
47 FROM company
48 JOIN transaction
49 ON company.id = company_id
50 WHERE declined = 0
51 ;
```

Result Grid

recuento_paises_generando_ventas
15

Output

#	Time	Action	Message	Duration / Fetch
100103	20:59:08	SELECT COUNT(DISTINCT country) AS recuento_paises_generando_ventas FROM company JOIN transaccio...	1 row(s) returned	0.234 sec / 0.000 sec

→ Identifica a la compañía con la mayor media de ventas.

```
55 • SELECT company.*, ROUND(AVG(amount),2) AS media_ventas
56 FROM company
57 JOIN transaction
58 ON company.id = company_id
59 WHERE declined = 0
60 GROUP BY company_id
61 ORDER BY media_ventas DESC
62 LIMIT 1
63 ;
```

Result Grid

id	company_name	phone	email	country	website	media_ventas
10-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site	284.91

Output

#	Time	Action	Message	Duration / Fetch
100176	12:19:21	SELECT company.*, ROUND(AVG(amount),2) AS media_ventas FROM company JOIN transaction ON company.id = company_id WHERE declined = ...	1 row(s) returned	0.218 sec / 0.000 sec

Ejercicio 3

Utilizando sólo subconsultas (sin utilizar JOIN):

→ Muestra todas las transacciones realizadas por empresas de Alemania.

```
70 SELECT transaction.*
71 FROM transaction
72 WHERE declined = 0 AND EXISTS (
73     SELECT company.id
74     FROM company
75     WHERE country = 'Germany'
76 )
77 ;
78
```

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00043A49-2949-494B-A5D0-A5BAE3B819D0	Cs-9294	b-2458	4713	46.1999	1.43554	2024-08-28 07:16:46	395.43	0
000447FE-B650-4D0F-85DE-C7ED0EE1CAAD	Cs-5019	b-2370	438	41.5972	12.2218	2016-12-21 20:07:18	155.63	0
00045D6B-ED2E-4F2F-8186-CEE074D875D0	Cs-6699	b-2390	2118	29.7573	-95.3796	2020-07-14 15:37:18	326.01	0
000481C3-1C26-4FEF-83A0-ACD0E004BBD0	Cs-6696	b-2230	2115	53.5489	-113.503	2017-09-04 19:44:53	161.60	0
00051AA4-9CBE-4268-8070-C38062A1B3E2	Cs-7606	b-2266	3025	52.2084	5.69081	2017-01-05 18:19:25	148.91	0

transaction 11 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
100111	21:25:26	SELECT transaction.* FROM transaction WHERE declined = 0 AND EXISTS (SELECT company.id FROM company WHERE country = 'Germany') ;	99763 row(s) returned	0.000 sec / 0.110 sec

→ Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones.

```
81 SELECT company_name
82 FROM company
83 WHERE EXISTS (
84     SELECT company_id
85     FROM transaction
86     WHERE declined = 0 AND amount > (
87         SELECT AVG(amount)
88         FROM transaction
89         GROUP BY company_id
90     )
91 )
92 ;
93
```

company_name
Ac Fermentum Incorporated
Magna A Neque Industries
Fusce Corp.
Conville In Incorporated
Ante Jaculis Nec Foundation

company 8 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
100178	12:56:11	SELECT company_name FROM company WHERE EXISTS (SELECT company_id FROM transaction WHERE declined = 0 AND amount > (SELECT AVG(amount) FROM transaction GROUP BY company_id)) ;	100 row(s) returned	0.031 sec / 0.000 sec

→ Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas empresas.

```
96 SELECT company.*
97 FROM company
98 WHERE NOT EXISTS (
99     SELECT company_id
100     FROM transaction
101 )
102 ;
103
```

id	company_name	phone	email	country	website
1	Ac Fermentum Incorporated	1234567890	info@acfermentum.com	USA	www.acfermentum.com
2	Magna A Neque Industries	9876543210	contact@magnaaneque.com	USA	www.magnaaneque.com
3	Fusce Corp.	5555555555	hello@fuscecorp.com	USA	www.fuscecorp.com
4	Conville In Incorporated	1111111111	info@convillein.com	USA	www.convillein.com
5	Ante Jaculis Nec Foundation	2222222222	contact@antejaculis.com	USA	www.antejaculis.com

company 8 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
100108	21:23:03	SELECT company.* FROM company WHERE NOT EXISTS (SELECT company_id FROM transaction) ;	0 row(s) returned	0.000 sec / 0.000 sec

Ejercicio 4

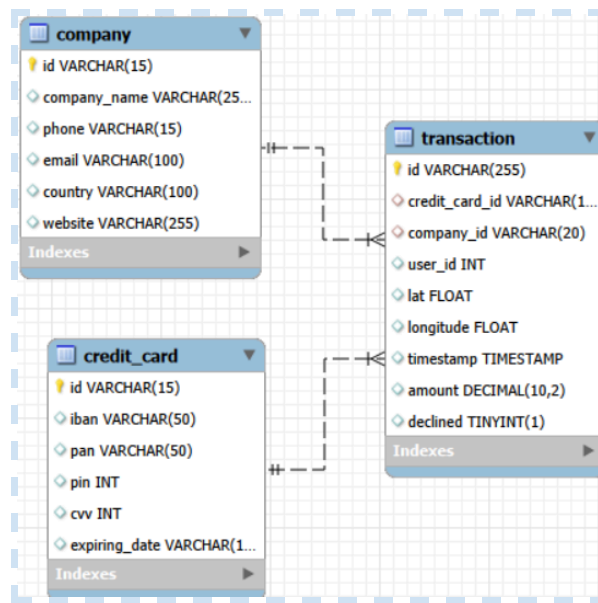
Tu tarea es diseñar y crear una tabla llamada **"credit_card"** que almacene detalles cruciales sobre las tarjetas de crédito. La nueva tabla debe ser capaz de identificar de forma única cada tarjeta y establecer una relación adecuada con las otras dos tablas (**"transaction"** y **"company"**). Después de crear la tabla será necesario que ingreses la información del documento denominado **"datos_introducir_credit.sql"**. Recuerda mostrar el diagrama y realizar una breve descripción del mismo.

The screenshot displays a SQL IDE interface with a script editor at the top and a results/output pane at the bottom. The script editor contains the following SQL commands:

```
106 • CREATE TABLE IF NOT EXISTS credit_card (  
107     id VARCHAR(15) PRIMARY KEY,  
108     iban VARCHAR(50),  
109     pan VARCHAR(50),  
110     pin INT,  
111     cvv INT,  
112     expiring_date VARCHAR(15)  
113 );  
114  
115 • ALTER TABLE transaction ADD CONSTRAINT credit_card_transaction  
116     FOREIGN KEY (credit_card_id) REFERENCES credit_card(id);  
117  
118 -- Insertar datos de 'N1-Ex-4_datos_introducir_credit.sql'  
119  
120 • UPDATE credit_card  
121     SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y');  
122  
123 • ALTER TABLE credit_card  
124     MODIFY expiring_date DATE;
```

The results/output pane shows the execution of these commands with the following details:

#	Time	Action	Message	Duration / Fetch
100113	22:59:51	CREATE TABLE IF NOT EXISTS credit_card (id VARCHAR(15) PRIMARY KEY, iban VARCHAR(50), pan VARCHAR(50), pin INT, cvv INT, expiring_date VARCHAR(15))	0 row(s) affected	0.016 sec
105255	18:25:25	ALTER TABLE transaction ADD CONSTRAINT credit_card_transaction FOREIGN KEY (credit_card_id) REFERENCES credit_card(id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.032 sec
105113	23:08:39	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9580', 'XX78125888985195080', '1234 5678 9010 1111', 1234, 5678, '2023-12-31')	1 row(s) affected	0.000 sec
105258	18:30:40	UPDATE credit_card SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y')	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0	0.078 sec
105259	18:30:43	ALTER TABLE credit_card MODIFY expiring_date DATE	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.062 sec



Se ha añadido la tabla `credit_card` al esquema anterior. Dicha tabla establece una relación 1:N con `transaction` a través de la foreign key `credit_card_id`.

La tabla `credit_card` incluye el IBAN y el PAN como cadenas que permiten varios caracteres (VARCHAR), el PIN y el CVV se guardan como números enteros y la fecha de expiración como VARCHAR también hasta una vez insertados los datos, que entonces se convierten a fecha (DATE).

Ejercicio 5

El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a su tarjeta de crédito con ID **CcU-2938**. La información que debe mostrarse para este registro es: **TR323456312213576817699999**. Recuerda mostrar que el cambio se realizó.

The screenshot shows a database management tool interface. The top panel displays an SQL statement: `UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2938';`. Below the statement, the 'Output' tab shows the execution results. A message indicates: '1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0'. The 'Duration / Fetch' is '0.000 sec'. Below the output, a 'Result Grid' is visible, showing the updated record for 'CcU-2938' with the new 'iban' value 'TR323456312213576817699999'. The bottom panel shows a 'SELECT * FROM credit_card WHERE id = 'CcU-2938'' statement, with a message indicating '1 row(s) returned' and a 'Duration / Fetch' of '0.000 sec / 0.000 sec'.

Ejercicio 6

En la tabla "**transaction**" ingresa una nueva transacción con la siguiente información:

Id	credit_card_id	company_id	user_id	lat	longitude	amount	declined
108B1D1D-5B23-A76C-55E F-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	111.11	0

The screenshot shows a database management tool interface. The top panel displays an SQL statement: `INSERT INTO credit_card (id) VALUES ('CcU-9999');`. Below the statement, the 'Output' tab shows the execution results. A message indicates: '1 row(s) affected'. The 'Duration / Fetch' is '0.016 sec'. Below the output, a 'Result Grid' is visible, showing the inserted record for 'CcU-9999'. The bottom panel shows a 'DELETE FROM transaction WHERE id = '108B1D1D-5B23-A76C-55EFC568E49A99DD'' statement, with a message indicating '0 row(s) affected' and a 'Duration / Fetch' of '0.000 sec'. Below that, another 'INSERT INTO transaction' statement is shown, with a message indicating '1 row(s) affected' and a 'Duration / Fetch' of '0.000 sec'.

Ejercicio 7

Desde recursos humanos te solicitan eliminar la columna "pan" de la tabla **credit_card**. Recuerda mostrar el cambio realizado.

The screenshot displays a database management interface with two main sections. The top section shows the execution of two SQL statements:

```
153 • ALTER TABLE credit_card DROP COLUMN pan;
154 • SELECT * FROM credit_card;
```

Below the SQL editor, the 'Output' pane shows the results of the executed statements:

#	Time	Action	Message	Duration / Fetch
105122	23:46:19	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) ...	1 row(s) affected	0.000 sec
105123	23:48:29	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec

The bottom section shows the 'Result Grid' for the second statement, displaying a table with 5 columns: id, iban, pin, cvv, and expiring_date. The table contains 5 rows of data:

id	iban	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	1819	467	09/27/25
CcS-4858	XX8581768137002436094025	3964	817	12/28/28
CcS-4859	XX7826930491423553609370	4983	277	11/26/26
CcS-4860	XX5559590368835304645299	6876	661	07/27/27
CcS-4861	XX2035182877195191627307	5710	398	04/25/26

The 'Output' pane below the result grid shows the execution of the second statement:

#	Time	Action	Message	Duration / Fetch
105125	23:50:03	SELECT * FROM credit_card	5001 row(s) returned	0.000 sec / 0.000 sec

Ejercicio 8

Descarga los archivos CSV que encontrarás en el apartado de recursos:

- *american_users.csv*
- *european_users.csv*
- *companies.csv*
- *credit_cards.csv*
- *transactions.csv*

Estudia y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas:

```
3 CREATE DATABASE IF NOT EXISTS Ex8;
4 USE Ex8;
5
6 CREATE TABLE IF NOT EXISTS companies (
7     company_id VARCHAR(15) PRIMARY KEY,
8     company_name VARCHAR(255),
9     phone VARCHAR(15),
10    email VARCHAR(100),
11    country VARCHAR(100),
12    website VARCHAR(255)
13 );
14
15 CREATE TABLE IF NOT EXISTS credit_cards (
16     id VARCHAR(255) PRIMARY KEY,
17     user_id VARCHAR(15) REFERENCES users(id),
18     iban VARCHAR(50),
19     pan VARCHAR(50),
20     pin INT,
21     cvv INT,
22     track1 VARCHAR(75),
23     track2 VARCHAR(75),
24     expiring_date CHAR(8)
25 );
26
27 CREATE TABLE IF NOT EXISTS users (
28     id INT PRIMARY KEY,
29     name VARCHAR(50),
30     surname VARCHAR(50),
31     phone VARCHAR(20),
32     email VARCHAR(100),
33     birth_date VARCHAR(13),
34     country VARCHAR(50),
35     city VARCHAR(50),
36     postal_code VARCHAR(10),
37     address VARCHAR(100)
38 );
39
40 CREATE TABLE IF NOT EXISTS transactions (
41     id VARCHAR(255) PRIMARY KEY,
42     card_id VARCHAR(15),
43     business_id VARCHAR(20),
44     timestamp TIMESTAMP,
45     amount DECIMAL(18, 2),
46     declined BOOLEAN,
47     product_ids VARCHAR(255),
48     user_id INT REFERENCES users(id),
49     lat FLOAT,
50     longitude FLOAT,
51     FOREIGN KEY (business_id) REFERENCES companies(company_id),
52     FOREIGN KEY (card_id) REFERENCES credit_cards(id),
53     FOREIGN KEY (user_id) REFERENCES users(id)
54 );
```

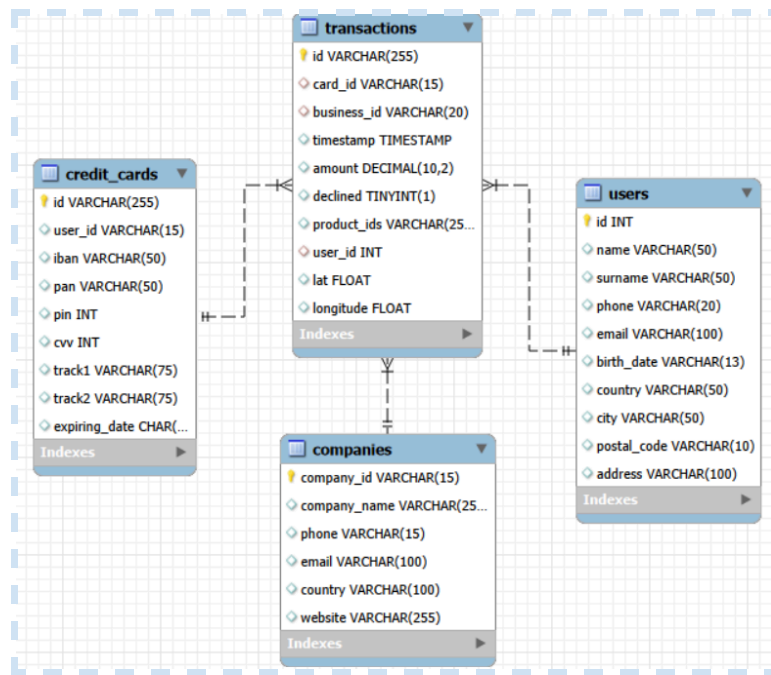
105261	18:52:24	CREATE DATABASE IF NOT EXISTS Ex8	1 row(s) affected	0.000 sec
105262	18:52:24	USE Ex8	0 row(s) affected	0.000 sec
105263	18:52:24	CREATE TABLE IF NOT EXISTS companies (company_id VARCHAR(15) PRIMARY KEY, company_name ...	0 row(s) affected	0.016 sec
105264	18:52:24	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(255) PRIMARY KEY, user_id VARCHAR(15)...	0 row(s) affected	0.000 sec
105265	18:52:24	CREATE TABLE IF NOT EXISTS users (id INT PRIMARY KEY, name VARCHAR(50), surname VARCH...	0 row(s) affected	0.016 sec
105266	18:52:24	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(255) PRIMARY KEY, card_id VARCHAR(15), b...	0 row(s) affected	0.015 sec

```

58 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__companies.csv'
59 INTO TABLE companies
60 FIELDS TERMINATED BY ','
61 LINES TERMINATED BY '\r\n'
62 IGNORE 1 LINES;
63
64 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__credit_cards.csv'
65 INTO TABLE credit_cards
66 FIELDS TERMINATED BY ','
67 LINES TERMINATED BY '\r\n'
68 IGNORE 1 LINES;
69
70 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__european_users.csv'
71 INTO TABLE users
72 FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY ''
73 LINES TERMINATED BY '\r\n'
74 IGNORE 1 LINES;
75
76 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__american_users.csv'
77 INTO TABLE users
78 FIELDS TERMINATED BY ',' ENCLOSED BY ''
79 LINES TERMINATED BY '\r\n'
80 IGNORE 1 LINES;
81
82 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__transactions.csv'
83 INTO TABLE transactions
84 FIELDS TERMINATED BY ','
85 LINES TERMINATED BY '\r\n'
86 IGNORE 1 LINES;

```

105267	18:56:13	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__companies.csv' INTO...	100 row(s) affected	Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.031 sec
105268	18:56:13	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__credit_cards.csv' INT...	5000 row(s) affected	Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.078 sec
105269	18:56:13	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__european_users.csv' I...	3990 row(s) affected	Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.078 sec
105270	18:56:14	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__american_users.csv' I...	1010 row(s) affected	Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.016 sec
105271	18:56:14	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__transactions.csv' INT...	100000 row(s) affected	Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	3.078 sec



Esquema de un modelo de tipo estrella con cuatro tablas; companies, transactions, users y credit_cards, usando ids VARCHAR como primary keys a excepción de users, que usa una compuesta por un número entero. La tabla de hechos es transactions y tiene una relación 1:N con las tablas de dimensiones; credit_cards, companies y users.

La tabla company incluye el nombre de la compañía, el teléfono, el correo electrónico, el país y la página web como cadena que permite varios tipos de caracteres (VARCHAR).

La tabla users incluye nombre, apellido, teléfono, correo electrónico, fecha de nacimiento, país, ciudad, código postal y dirección todos como VARCHAR.

La tabla credit_cards incluye la id de usuario, que referencia a la tabla de usuarios, el IBAN, el PAN y las pistas 1 y 2 como VARCHAR, el PIN y el CVV se guardan como números enteros y la fecha de expiración como cadena con un número específico de caracteres (CHAR).

Por último, la tabla transactions incluye la id de la tarjeta de crédito, la compañía y los productos como VARCHAR, la fecha y hora de la transacción como TIMESTAMP, la cantidad como decimal exacto (DECIMAL), si se ha rechazado o no la transacción, como un valor binario (TINYINT), la id de usuario como número entero (INT) y latitud y longitud como decimal aproximado (FLOAT).

Ejercicio 9

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

```
91 • SELECT *
92 FROM users
93 WHERE EXISTS (
94     SELECT transactions.user_id
95     FROM transactions
96     WHERE user_id = users.id AND declined = 0
97     GROUP BY transactions.user_id
98     HAVING COUNT(*) > 80
99 );
```

id	name	surname	phone	email	birth_date	country	city	postal_code	address
185	Molly	Gilliam	0800 120 8023	donec@outlook.couk	Dec 21, 1993	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 MI Rd.
289	Dxwgi	Hwcru	+98-309-8797	dxwgi.hwcru@example.com	Aug 20, 1976	Germany	Stuttgart	70173	82 Hwcru Street
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	May 3, 1974	Italy	Genoa	16100	53 Astuw Street

105272 19:16:57 SELECT * FROM users WHERE EXISTS (SELECT transactions.user_id FROM transactions WHERE u... 3 row(s) returned 0.219 sec / 0.000 sec

Ejercicio 10

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.

```
103 • SELECT iban, ROUND(AVG(amount),2) AS cantidad_media
104 FROM transactions
105 JOIN credit_cards
106     ON card_id = credit_cards.id
107 JOIN companies
108     ON business_id = company_id
109 WHERE company_name LIKE 'Donec Ltd'
110 GROUP BY credit_cards.iban
111 ;
```

iban	cantidad_media
XX911406401125586307586805	356.25
SK9446370242474562577506	142.96
XX776752917845952975555640	257.37
XX413827362289719304908990	139.59
XX347787246070769610780308	240.41
XX688768436543090894854602	188.58
MK28368851538688349	439.39
PL76249283566852676343404576	541.56
XX804008331134918341803913	189.21
LB6465553777363327873049938	155.50
XX911406401125586307586805	356.25

105274 19:20:49 SELECT iban, ROUND(AVG(amount),2) AS cantidad_media FROM transactions JOIN credit_cards ON card_id... 371 row(s) returned 0.000 sec / 0.000 sec

Nivel 2

Ejercicio 1

Identifica los cinco días que se generó la mayor cantidad de ingresos en la empresa (supuestamente Donec Ltd) por ventas. Muestra la fecha de cada transacción junto con el total de las ventas.

```
117 • SELECT SUM(amount) AS ingresos, DATE(timestamp) AS fecha
118 FROM transactions
119 JOIN companies
120 ON business_id = company_id
121 WHERE company_name LIKE 'Donec Ltd'
122 GROUP BY DATE(timestamp)
123 ORDER BY SUM(amount) DESC
124 LIMIT 5
125 ;
```

ingresos	fecha
1624.68	2022-12-13
1381.74	2018-11-26
1126.46	2015-05-09
1044.10	2015-12-04
915.09	2024-10-25

Result 4 x

Output

#	Time	Action	Message	Duration / Fetch
93	00:33:01	SELECT ban, AVG(amount) AS cantidad_medio FROM transactions JOIN credit_cards ON card_id = credit_c...	371 row(s) returned	0.015 sec / 0.000 sec
94	00:34:16	SELECT SUM(amount) AS ingresos, DATE(timestamp) AS fecha FROM transactions JOIN companies ON busi...	5 row(s) returned	0.016 sec / 0.000 sec

Ejercicio 2

Presenta el nombre, teléfono, país, fecha y amount, de aquellas empresas que realizaron transacciones con un valor comprendido entre 350 y 400 euros y en alguna de estas fechas: 29 de abril de 2015, 20 de julio de 2018 y 13 de marzo de 2024. Ordena los resultados de mayor a menor cantidad.

```
130 • SELECT company_name, phone, country, timestamp, amount
131 FROM companies
132 JOIN transactions
133 ON company_id = business_id
134 WHERE amount BETWEEN 350 AND 400
135 AND DATE(timestamp) IN ('2015-04-29', '2018-07-20', '2024-03-13')
136 ORDER BY amount DESC
137 ;
```

company_name	phone	country	timestamp	amount
Aliquam PC	01 45 73 52 16	Germany	2024-03-13 01:07:21	399.84
Auctor Mauris Vel LLP	08 09 28 74 14	United States	2018-07-20 13:57:29	399.51
At Pede Corp.	06 14 48 33 15	Italy	2015-04-29 15:35:40	390.69
Aliquam PC	01 45 73 52 16	Germany	2024-03-13 19:05:41	388.29
Orci Adipiscing Limited	03 18 00 77 81	United Kingdom	2018-07-20 01:20:14	373.71
Fringilla LLC	08 29 15 93 57	New Zealand	2015-04-29 06:18:25	367.62
Quam Praesent Ltd	07 67 56 48 38	Norway	2018-07-20 14:57:17	366.87

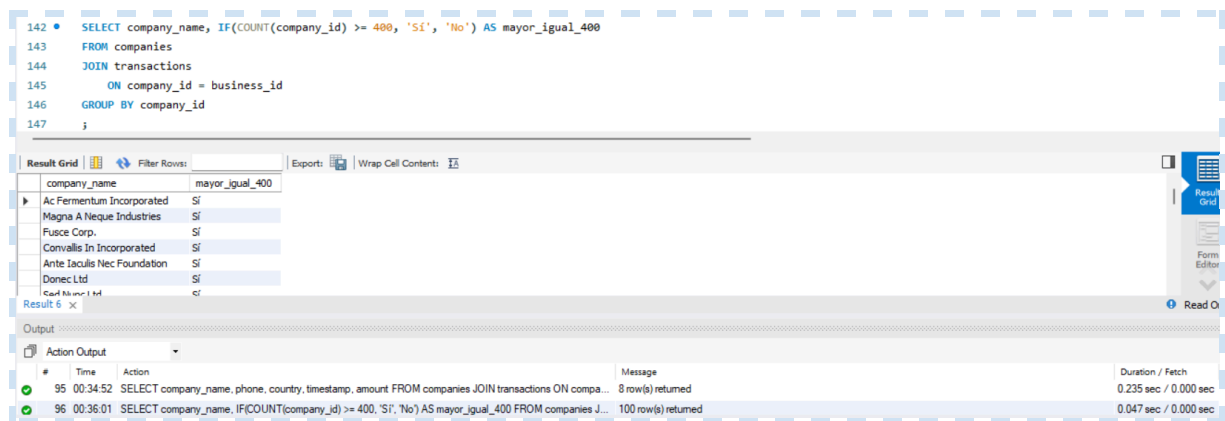
Result 5 x

Output

#	Time	Action	Message	Duration / Fetch
94	00:34:16	SELECT SUM(amount) AS ingresos, DATE(timestamp) AS fecha FROM transactions JOIN companies ON busi...	5 row(s) returned	0.016 sec / 0.000 sec
95	00:34:52	SELECT company_name, phone, country, timestamp, amount FROM companies JOIN transactions ON compa...	8 row(s) returned	0.235 sec / 0.000 sec

Ejercicio 3

Necesitamos optimizar la asignación de los recursos y dependerá de la capacidad operativa que se requiera, por lo que te piden la información sobre la cantidad de transacciones que realizan las empresas, pero el departamento de recursos humanos es exigente y quiere un listado de las empresas en las que especifiques si tienen igual o más de 400 transacciones o menos.



```
142 • SELECT company_name, IF(COUNT(company_id) >= 400, 'SI', 'No') AS mayor_igual_400
143 FROM companies
144 JOIN transactions
145 ON company_id = business_id
146 GROUP BY company_id
147 ;
```

company_name	mayor_igual_400
Ac Fermentum Incorporated	SI
Magna A Neque Industries	SI
Fusce Corp.	SI
Convalis In Incorporated	SI
Ante Iaculis Nec Foundation	SI
Donec Ltd	SI
Card Nunc Ltd	SI

Result 6 x

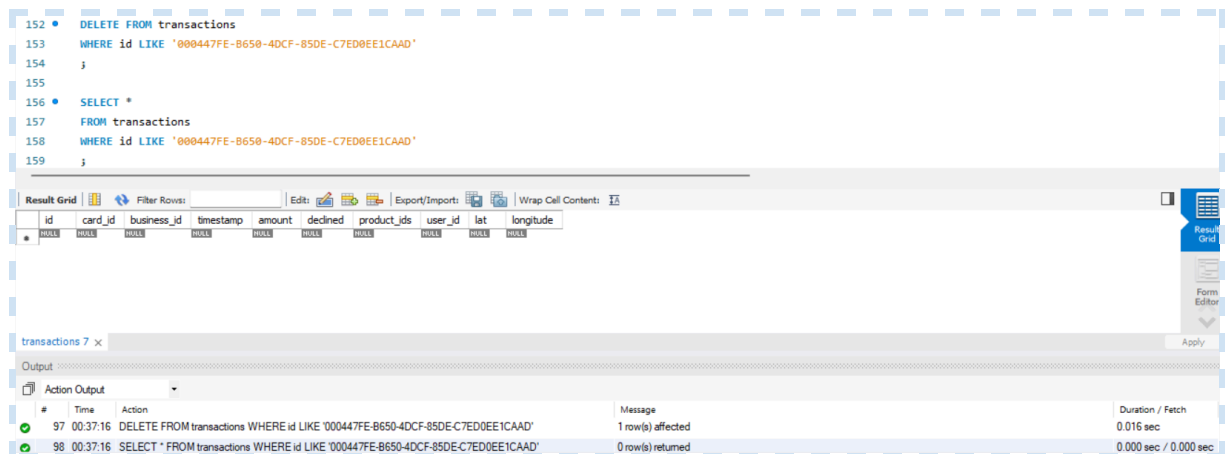
Output

Action Output

#	Time	Action	Message	Duration / Fetch
95	00:34:52	SELECT company_name, phone, country, timestamp, amount FROM companies JOIN transactions ON compa...	8 row(s) returned	0.235 sec / 0.000 sec
96	00:36:01	SELECT company_name, IF(COUNT(company_id) >= 400, 'SI', 'No') AS mayor_igual_400 FROM companies J...	100 row(s) returned	0.047 sec / 0.000 sec

Ejercicio 4

Elimine el registro con ID **000447FE-B650-4DCF-85DE-C7ED0EE1CAAD** de la tabla de transacciones.



```
152 • DELETE FROM transactions
153 WHERE id LIKE '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'
154 ;
155
156 • SELECT *
157 FROM transactions
158 WHERE id LIKE '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'
159 ;
```

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
...	...	...	...	...	...	...	...	...	...

transactions 7 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
97	00:37:16	DELETE FROM transactions WHERE id LIKE '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected	0.016 sec
98	00:37:16	SELECT * FROM transactions WHERE id LIKE '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	0 row(s) returned	0.000 sec / 0.000 sec

Ejercicio 5

La sección de marketing desea tener acceso a información específica para realizar análisis y estrategias efectivas. Se ha solicitado crear una vista que proporcione detalles clave sobre las compañías y sus transacciones. Será necesaria que crees una vista llamada **VistaMarketing** que contenga la siguiente información: Nombre de la compañía. Teléfono de contacto. País de residencia. Media de compra realizado por cada compañía. Presenta la vista creada, ordenando los datos de mayor a menor promedio de compra.

```
164 • CREATE OR REPLACE VIEW VistaMarketing AS
165 SELECT company_name, phone, country, ROUND(AVG(amount), 2) AS media_compra
166 FROM companies
167 JOIN transactions
168 ON company_id = business_id
169 GROUP BY company_id
170 ORDER BY media_compra DESC
171 ;
```

105278 19:28:13 CREATE OR REPLACE VIEW VistaMarketing AS SELECT company_name, phone, country, ROUND(AVG(a... 0 row(s) affected 0.000 sec

company_name	phone	country	media_compra
Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.87
Pretium Neque Corp.	07 77 48 55 28	Australia	276.16
Urn a Convalis Associates	06 01 24 77 04	United States	274.24
At Associates	09 56 61 10 65	New Zealand	272.21
Metus Vitae Associates	08 25 44 40 66	Australia	270.08
Aliquet Diam Limited	02 76 61 47 46	United States	269.60
Nec Luctus LLC	02 14 71 75 73	Norway	268.60
Neque Tellus Incorporated	04 43 18 34 19	Ireland	267.85
Tortor Nunc Commodo Company	05 35 92 77 16	United States	267.84
Cras Consulting	07 50 10 85 63	Belgium	267.44
Fruct Insom Ltd	03 67 44 56 72	United States	267.06

105279 19:29:52 SELECT * FROM ex8.vistamarketing 100 row(s) returned 0.203 sec / 0.000 sec

